

Úvod

Git je verzovací systém pro projekty. Umožňuje snadnou spolupráci v týmu a diferenciální zálohování. Umí výborně pracovat s textovými soubory, s binárními ale efektivně pracovat neumí (např. obrázky, exe soubory apod.).

Verzovací systém, jak název napovídá, umožňuje vytvářet verze souborů (projektů)

a následně verze spojovat a vracet se k minulým. Verzovacích systémů existuje vícero – např. Mercurial, CVS, Subversion. Git patří mezi nejrozšířenější díky jeho jednoduchosti a efektivitě – a je zdarma.

S Gitem se dá pracovat čistě konzolově (*git bash*) nebo za pomoci uživatelského rozhraní (*git UI*) – při instalaci na Windows se zpravidla nainstalují obě možnosti. Dále je možné použít další programy – doporučuji program *Sourcetree* (máme v učebnách), který je o něco přehlednější a také zdarma. Dále je standardem, že vývojová prostředí mají základní rozhraní Gitu zabudované, např. JetBrains produkty a VS Code.

Diferenciální zálohování vytváří verze souborů jen jako množinu změn, např. přidaná řádka, přepsané slovo, vytvořený nový soubor, smazaná složka apod. Tudiž na rozdíl

od hrubého vytváření kompletních záloh (např. každý den vytvoření nového zip archivu celého projektu) má zdaleka menší nároky na paměť. Bývá také rychlejší při vracení se k jiným verzím – systém nemusí přepisovat tak velké množství souborů.

K používání Gitu je potřeba si nastavit uživatelské údaje – uživatelské jméno a email. Tyto údaje slouží jako váš podpis – přidávají se jako metadata ke každé verzi projektu, kterou vytvoříte.

Důležitá vlastnost Gitu je, že každý má svoji kopii projektu. Hlavním úložištěm bývá nějaký server (buď vlastní, nebo systém *Gitlab* nebo *Github* – ve škole máme pro projekty vytvořený školní Gitlab, na kterém budete na konci studia pravděpodobně odevzdávat svoji maturitní práci).

Server udává „společný stav“ projektu; každý nový uživatel přebírá projekt odsud. Uživatelé pak ale pracují se svojí lokální kopií, kterou po dokončení své práce (vytvoření nové verze) nahrají zpět na server. Ostatní uživatelé si ji pak mohou stáhnout jako novou verzi – ale rozpracovaná práce těchto uživatelů se nijak nezmění.

Výhodou vícero kopií projektu je, že když server vypadne, uživatelé mohou pracovat bez potíží dál na své kopii. Pokud se server dokonce smaže, může libovolný uživatel vytvořit úložiště serveru znovu ze svojí kopie. Žádná práce se tedy nemusí ztratit.

Repositář

Úložiště každého projektu se označuje termínem repositář. Slouží jako prostor pro ukládání verzí projektu. Git pro tyto účely přidává do projektu skrytou složku „*git*“.

Repositář je buď lokální nebo vzdálený. Vzdálený je společným úložištěm pro všechny uživatele. Lokální repositář je kopie vzdáleného repositáře na počítači každého uživatele. Uživatelé v lokálním repositáři pracují (vytváří změny) a ty následně nahrávají zpět na vzdálený repositář, aby si je ostatní uživatelé mohli vyzvednout.

Verze (commit)

Uživatelé při práci vytváří verze projektu. Zabalí všechny změněné soubory, které vyberou, do jednoho balíčku (commit) a označí ho popisem verze (např. „přidán systém pro přihlašování uživatelů“). Tím se vytvoří nová verze pouze na lokálním repositáři tohoto uživatele. Ostatní uživatelé o těchto změnách neví, dokud je daný uživatel nenahraje na společný vzdálený repositář. I poté se projekt ostatních uživatelů nezmění – stále pracují se svou kopií projektu. Aby si změny vyzvedli, musí si danou verzi stáhnout do svého projektu (příkaz Fetch) a aplikovat ji (příkaz Checkout/Pull). Tím se spojí jejich rozpracovaná práce s nově nahranou verzí druhého uživatele.

Větev (branch)

Další výhodou systému Git je větvení projektu. Větvení znamená, že může paralelně běžet vývoj několika verzí projektu (zpravidla např. „vývojová, prerelease, release a stable“). Další možnost pro větvení projektu je, že každý uživatel má svoji větev, ve které pracuje, toto je zpravidla použitelné jen pro malé týmy.

Verze projektu (commity) se vždy nahrávají do vybrané větve. Každý projekt má alespoň jednu větev a vždy je jedna větev zvolena jako hlavní větev – ta, kterou si stáhnou noví uživatelé při stáhnutí kopie celého projektu (příkaz Clone).

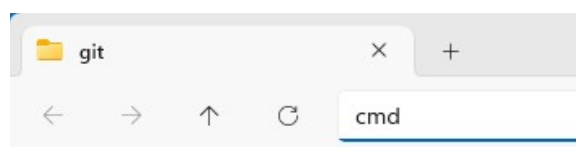
Větve se následně dají spojovat dohromady. Např. mám větve „vývoj“ a „veřejná“, do vývoje nahrají vývojáři postupně 10 commitů, posledním commitem mám vytvořenou novou veřejně publikovatelnou verzi projektu – spojím tedy obě větve do společného stavu, a tím se do veřejné větve připojí všechny doposud vytvořené commity vývojové větve (bez toho, aby v ní někdy byla rozpracovaná práce vývojové větve).

Stejným způsobem lze spojovat i paralelní vývoj. Např. ve větvi „vývoj“ mám 5 commitů, ve větvi „bugfixy“ mám 3 commity. Všechny tyto commity spojím do jednoho společného commitu (příkazem Merge), kterým se obě větve dostanou do společného bodu, ze kterého mohou opět vycházet dále ve vývoji.

Terminál

Terminál otevřu jako program *cmd* při vyhledávání v aplikacích systému.

Rychlý způsob pro otevření terminálu ve složce, kterou mám otevřenou v průzkumníku souborů je, že kliknu nahoře do pole cesty (path) a přepíšu ji na *cmd* a stisknu Enter.



K procházení složek v terminálu slouží příkaz „cd“. Např. „cd C:\Users“ (absolutní cesta) změní aktuální složku terminálu do „C:\Users“. Relativní cesta se zadává bez disku, např. „cd temp“ změní aktuální složku terminálu na podsložku „temp“ aktuální složky.

Všechny příkazy systému Git se píší do příkazové řádky za příkaz „git“. Tudíž příkaz „init“ zavolám v příkazové řádce jako „git init“.

Nastavení uživatelského profilu (config)

Jak bylo zmíněno v úvodu, ke každé verzi projektu (commitu) se přidávají metadata o autorovi – jeho jméno a email. Tyto údaje si uživatel musí nastavit před tím, než začne vytvářet verze v projektu:

```
C:\Users\hladilova\git> git config user.email hladilova@spseplzen.cz
```

```
C:\Users\hladilova\git> git config user.name "Tammie Hladilová"
```

Tyto příkazy nastaví uživatelské údaje pouze pro aktuální repozitář. Pokud chci nastavit údaje pro všechny repozitáře na daném zařízení (vhodné pro osobní nebo pracovní počítače), stačí přidat přepínač „--global“, tudíž „git config --global user.name <jmeno>“.

K zjištění aktuálních uživatelských údajů v daném repozitáři slouží přepínač „--get“, tudíž „git config --get user.email“.

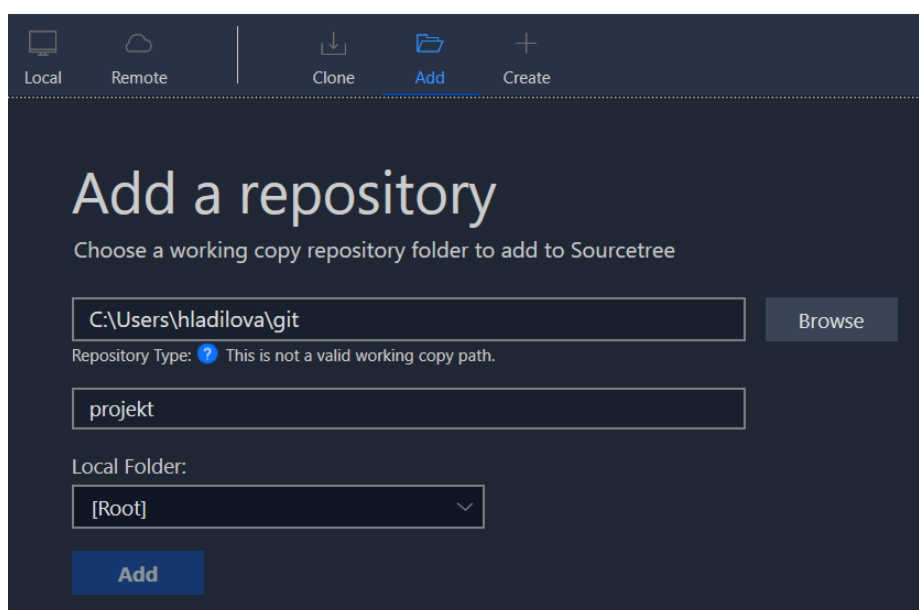
Žádný z údajů se automaticky nekontroluje, záleží na nás, aby byly údaje správné. Slouží převážně pro komunikaci mezi lidmi, aby ostatní věděli, jak autora kontaktovat.

Vytvoření projektu (init, clone)

Vytvoření nového prázdného lokálního projektu ve složce, ve které mám otevřený terminál, se provede pomocí příkazu „git init“. Toto v aktuální složce vytvoří nový „git“ adresář s potřebnými daty:

```
C:\Users\hladilova\git> git init  
Initialized empty Git repository
```

Takto vytvořený projekt mohu přidat (anebo rovnou vytvořit) přes grafické rozhraní (záložka Add, respektive Create):



Zkopírování projektu ze vzdáleného repozitáře do složky, ve které mám otevřený terminál, udělám pomocí příkazu „git clone“. Tím se do složky nakopírují všechny důležité informace o daném repozitáři a nastaví se mi verze projektu (současný commit – HEAD) dle posledního commitu výchozí větve projektu. Parametrem příkazu je URL repozitáře.

```
C:\Users\hladilova\git> git clone https://github.com/SanderMertens/flecs  
Cloning into 'flecs'..  
remote: Enumerating objects: 84762, done.  
remote: Counting objects: 100% (2215/2215), done.  
remote: Compressing objects: 100% (899/899), done.  
remote: Total 84762 (delta 1771), reused 1501 (delta 1316), pack-reused 82547  
(from 5)  
Receiving objects: 100% (84762/84762), 160.35 MiB | 1.82 MiB/s, done.  
Resolving deltas: 100% (61889/61889), done.
```

Zajímavé údaje z výpisu jsou velikost projektu a celkový počet verzovaných souborů.

Stejně tak je možné klonovat repositář přes grafické rozhraní – záložka Clone.

Vytvoření verze projektu (add, diff, status, commit)

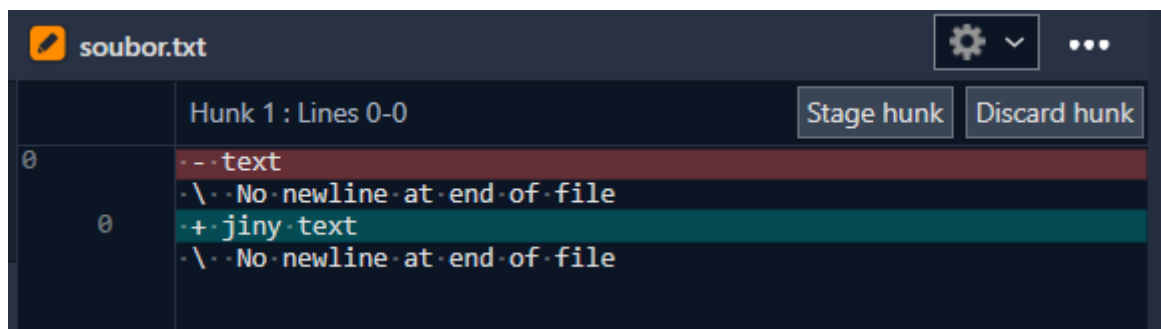
Vyberu všechny soubory, které chci nahrát jako součást nové verze projektu, pomocí příkazu „**add**“. Např. „git add config.txt“ přidá změny souboru „config.txt“ do nové verze. Příkaz „git add *“ přidá změny všech souborů do nové verze – používat opatrně, do verzí nechceme přidávat nepotřebné soubory – budou zbytečně zabírat paměť a zpomalovat manipulaci s verzemi. Opakem příkazu add je příkaz „git reset [<soubor>]“.

Před tím, než změny v souboru připravím k přidání do nové verze, mám možnost si zobrazit, jaké změny se ve vybraném souboru staly, příkazem „git **diff** [<soubor>]“.

```
C:\Users\hladilova\git>git diff
diff --git a/soubor.txt b/soubor.txt
index f3a3485..a8656a5 100644
--- a/soubor.txt
+++ b/soubor.txt
@@ -1,1 @@
-text
+jiny text
```

Vynecháním parametru <soubor> se vypíše změny všech souborů:

První řádek (---) značí výchozí soubor před změnami. Druhý řádek (+++) značí výchozí soubor s již provedenými změnami. Třetí řádek (@@) označuje kolik změn nastalo – zde 1 řádka odebrána, 1 řádka přidána. Zbytek řádek vypisuje dané změny v souboru – červené řádky uvozené znakem (-) označují odstraněné řádky, zelené řádky uvozené znakem (+) uvozují řádky přidané. Všimněte si, že změněná řádka = odstraněná řádka + přidaná řádka.



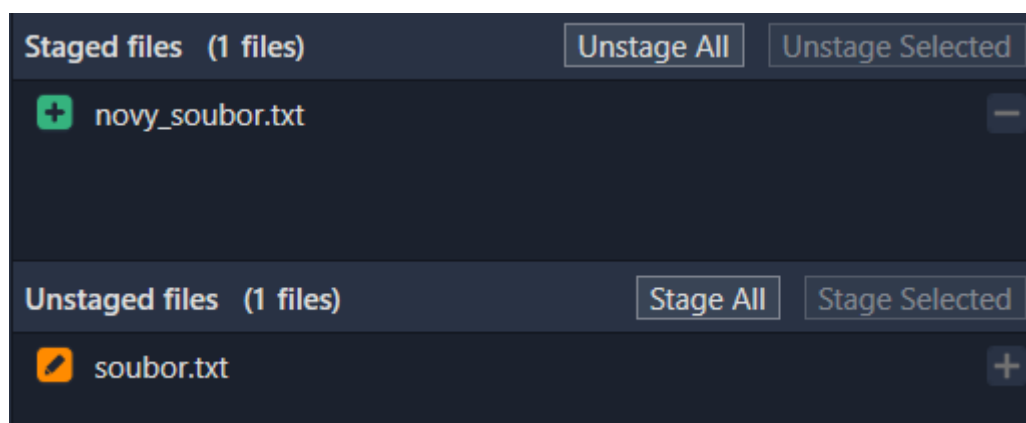
V grafickém rozhraní jsou dvě tlačítka – „Stage hunk“ – stejné jako „git add“ na vybraný text, a „Discard hunk“ – zahodí změnu (vrátí část souboru do původního stavu) stejně jako „git restore“.

Příkaz **status** přehledně vypíše všechny změny připravené i nepřípravené do nové verze – navíc i vytvořené a smazané soubory:

```
C:\Users\hladilova\git>git status
On branch master
Changes to be committed:
  new file:   novy_soubor.txt

Changes not staged for commit:
  modified:   soubor.txt
```

V první části výpisu jsou tzv. „staged“ změny – změny připravené do commitu, v druhé části změny, které nebyly příkazem add přidány – tzv. „unstaged“ změny. Ve výpisu je stručně popsán i druh změny – vytvořen/smazán/změněn.



Jakmile mám příkazem add vybrané všechny změněné soubory, které chci dát do nové verze, zavolám příkaz „commit“, který změny souborů zabalí a verzi uzavře. Parametrem příkazu je slovní popis změn. Tudiž volám například „git

commit -m "*oprava práv administrátora při vzdáleném přístupu*"“, kde přepínač „-m“ udává slovní popis změn v daném commitu – povinný parametr.

Každému commitu je přiřazen unikátní identifikátor – náhodný hexadecimální hash délky 40 znaků. Pro přehlednost (a díky množství unikátních hashů) se uvádí jen prvních několik znaků – identifikace prvními znaky (prefixem) je téměř zaručeně shodná s použitím celého hashe.

Hash může například vypadat takto:

756e2087d252cdca75ede6517bec54ca105e8215, jeho zkrácená verze jako 756e208. Délka zkrácené verze se může lišit – systém jen potřebuje rozpoznat, o který hash se původně jednalo, tudíž v malém projektu je možné klidně používat jen první tři znaky.

Pro vybrání verze projektu (příkaz Checkout) totiž zadáváme hash daného commitu, který chceme nastavit naší lokální kopii projektu. Alternativou je zadání názvu větve – tím se vybere nejnovější commit dané větve bez nutnosti hledat hash.

Ignorování souborů a složek

Ne všechny soubory chceme zálohovat. Např. dočasné soubory, logovací záznamy, obsahy knihoven třetích stran, nastavení prostředí daného uživatele apod.

Můžeme automaticky zamezit, aby některé soubory nebo typy souborů či složky bylo možné zálohovat. Toto se zajišťuje pomocí souboru „.gitignore“ – pozor na tečku na začátku názvu.

Do gitignore souboru se po řádkách zapisují jako výrazy podobné *Regex* výrazům označující soubory či složky, které nechceme zálohovat. Výrazy se vyhodnocují cestou relativně k umístění gitignore souboru a testují se pouze názvy souborů a složek (jejich obsah ne). Gitignore souborů může být v projektu vícero.

Příklad souboru [.gitignore]:


```

aplikace.exe
*.txt
.vscode
**/*.log
out/**/temp/*

```

Takovýto soubor určuje, že se nebudou zálohovat:

- Soubor „aplikace.exe“ v této složce
- Všechny textové soubory (přípona „.txt“) v této složce
- Složka „.vscode“ a její obsah
- Všechny „.log“ soubory v této složce a libovolné podsložce
- Složky „temp“ a jejich obsah v libovolné podsložce podsložky „out“

Větvě a commity projektu (log)

Příkazem „git log“ si můžeme vypsat commity daného projektu (klávesa „Q“ pro ukončení výpisu). V konzoli ale takový výpis nemusí být přehledný, zde se hodí použít aplikaci s uživatelským rozhraním pro Git (v příkladech Sourcetree).

Graph	Description	Date	Author	Commit
	#345 update DeferredActions testcase	21 bře 2021 7:05	Sander Mertens <s	929f3109f
	#345 replace CHILDOF with EcsChildOf in core	21 bře 2021 5:34	Sander Mertens <s	a61a3628
	Merge pull request #352 from Hexlord/patch-1	20 bře 2021 23:17	Sander Mertens <s	7202e07b
	Fix CMake warning	20 bře 2021 17:01	Alexander Knorre <	30bb5bb:
	Merge remote-tracking branch 'origin/master' into relation_api	20 bře 2021 2:48	Sander Mertens <s	5bb7223e
	#124 add assert to ecs_new_entity to ensure world is not a stage	20 bře 2021 2:47	Sander Mertens <s	f0eeeff14
	#124 use actual world when getting pipeline	19 bře 2021 20:18	Sander Mertens <s	3078eef8i
	#124 fix issue where overriding a tag could trigger assert	19 bře 2021 19:34	Sander Mertens <s	c7f8219b
	Merge remote-tracking branch 'origin/master' into relation_api	19 bře 2021 19:03	Sander Mertens <s	5e8936ca
	#124 update staging flow diagram	19 bře 2021 17:26	Sander Mertens <s	fd4e6b37
	#350 fix issue with creating readonly children iterator	18 bře 2021 20:46	Sander Mertens <s	47bc0db:
	#350 allow for converting entity handles to mutable	18 bře 2021 20:12	Sander Mertens <s	24f242f4f
	#349 fix rematching CASCADE query w/empty table	18 bře 2021 11:15	Sander Mertens <s	1ca5ce71
	#345 further deprecate old trait API	18 bře 2021 10:46	Sander Mertens <s	ae744d7c
	#322 remove STL types	18 bře 2021 2:59	Sander Mertens <s	4525769a
	Merge pull request #348 from randy408/get_world	17 bře 2021 20:12	Sander Mertens <s	213f4ea9f
	ecs_get_world(): update param description	17 bře 2021 20:09	Randy <randy408@	8e9135e1
	make ecs_get_world() public	17 bře 2021 19:28	Randy <randy408@	c61133a7
	#345 deprecate C++ trait API	17 bře 2021 10:53	Sander Mertens <s	ee75a94f
	#345 update C API to new terminology	17 bře 2021 6:26	Sander Mertens <s	a0f4e414
	Merge pull request #342 from SanderMertens/cpp_api_split_header	16 bře 2021 4:37	Sander Mertens <s	03ebb8f5
	#341 update amalgamated source	16 bře 2021 3:14	Sander Mertens <s	fe87af562
	#341 split up C++ API into multiple files	16 bře 2021 3:03	Sander Mertens <s	6b73a389
	#339 add macro that suppresses deprecated warnings	15 bře 2021 2:54	Sander Mertens <s	3fe055eb
	Merge pull request #335 from SanderMertens/staging_api	15 bře 2021 2:43	Sander Mertens <s	9fc7d48e

Vlevo je vidět graf commitů v rámci větví (každá lomená čára je větev, každá tečka je commit). Jsou zde vidět i commity, které spojují stavy dvou větví dohromady (tečky, kde se čáry spojují). Dále jsou vidět výchozí commity pro vytvořené větve (tečky, ze kterých vychází nová čára).

Dále je popis, datum vytvoření, autor (jméno a email) a zkrácený hash daného commitu.

Nastavení verze (checkout)

Příkazem **checkout** si nastavím v lokálním repozitáři verzi projektu, kterou chci nyní používat. Příkaz má dvě možnosti – buď nastavit verzi dle větve, nebo na specifický commit mimo aktuální stav větve – preferujte nastavování verze dle větve.

Formátem je „git checkout <větev/hash_commitu>“. Pro vykonání příkazu nesmí být v projektu rozpracovaná práce – tudíž nutno buď změny uložit (commit), schovat (stash), nebo smazat (discard). Pokud použijeme variantu s větví, nastaví se nám aktuální větev na tu, kterou „checkoutujeme“.

V případě, že použijeme variantu se specifickým commitem a daný commit není nejnovější commit ve své větvi, nastaví se nám projekt do tzv. stavu „**detached HEAD**“, což je stav, kdy nemáme nastavenou žádnou aktuální větev a nemůžeme tedy ukládat žádné změny. Tento stav se hodí pro prohlížení rozpracovaných verzí větví. Ze stavu detached HEAD se pro práci s projektem musíme dostat pryč – buď checkout do nějaké větve (git checkout <větev>) nebo vytvořením nové větve v daném commitu (git branch <název>).

Práce se vzdáleným repozitářem (remote)

Vzdálených repozitářů (tzv. „remote“) můžeme mít nastavených vícero, jeden z nich je ale vždy označen jako výchozí (zpravidla pojmenován jako „origin“). Zpravidla ale mají projekty nastaven jen jeden vzdálený repozitář.

Nastavení vzdáleného repozitáře našemu lokálnímu projektu (vytvořeným pomocí „git init“) se dělá příkazem **remote**. Formát příkazu je následující „git remote <podpříkaz>“. Hlavní podpříkazy jsou „add“ a „remove“. Pokud jsme pro vytvoření projektu použili příkaz clone, bude výchozí remote již nastaven automaticky.

Přidání remotu pomocí „git remote add <název_remotu> <URL_remotu>“ - např. „git remote add origin https://github.com/SanderMertens/flecs“.

Odstranění remotu pomocí „git remote remove <název_remotu>“ - např. „git remote remove origin“.

Verzování se vzdáleným repozitářem (push, fetch, pull)

Pro práci se vzdáleným repozitářem se používají tři příkazy – Push, Fetch a Pull.

Příkaz **push** nahraje lokální změny v zadané větvi na vzdálený repozitář, aby je viděli i ostatní uživatelé. Formátem příkazu je „git push <remote> <větev>“.

Příkaz **fetch** aktualizuje dle vzdáleného repozitáře stavy verzí a větví v projektu – informace o nových nebo smazaných větvích či commitech od ostatních uživatelů. Se stavem lokálního repozitáře tento příkaz nic nedělá – všechny lokální soubory zůstanou beze změny.

Příkaz má formát: „git fetch <remote> [<větev>]“, kde remote, je název vzdáleného repozitáře a nepovinný parametr větev určuje, kterou větev chceme aktualizovat, pokud nevyplněno, aktualizují se všechny větve. Užitečné argumenty příkazu jsou navíc „--force“ (vynutit i přes chyby), „--tags“ (aktualizovat i tagy), „--prune“ (smazat lokální tagy/větve, které již na remotu neexistují – někdo je smazal).

Příkaz **pull** je spojením příkazů „fetch <větev>“ a „checkout <větev>“. Tudíž aktualizuje stav dané větve dle vzdáleného repozitáře a ihned nastaví lokální repozitář do stavu nejnovějšího commitu dané větve. Stejně jako u samostatného příkazu checkout je nutné, aby v projektu nebyly rozpracované změny.

Odkládání a mazání změn (stash, restore)

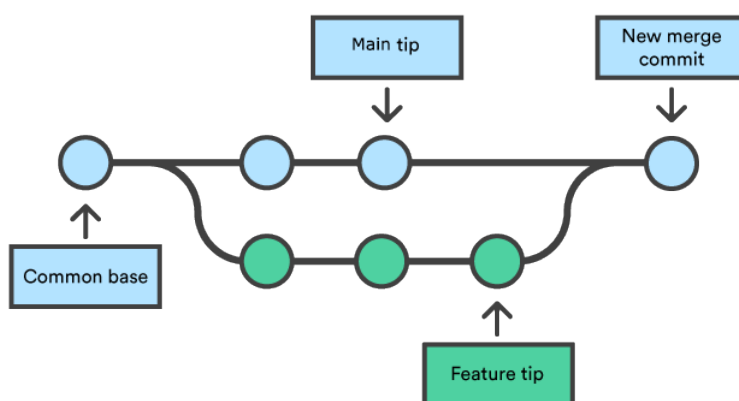
Pro zahození změn ve vybraném souboru nebo ve všech souborech slouží příkaz „git restore <soubor>“. V grafickém rozhraní tlačítka „Discard“ na samotných souborech nebo v horní liště pro výběr vícero souborů.

Pro zabalení a schování změn slouží příkaz „git stash push <název_balíčku>“. Příkaz vezme všechny staged soubory (příkazem add) a zabalí je do lokálního pojmenovaného balíčku změn a změny schová. Z projektu se tedy tyto změny smažou, ale máme možnost je aplikovat zpět příkazem „git stash pop“. Tím se všechny zabalené změny aplikují opět do projektu a balíček změn se smaže.

Spojování větví (merge, rebase)

Při spojování větví mohou nastat tři situace:

1. Jedna z větví nemá žádné změny
2. Obě větve mají vůči sobě nekonfliktní změny
3. Obě větve mají konfliktní změny

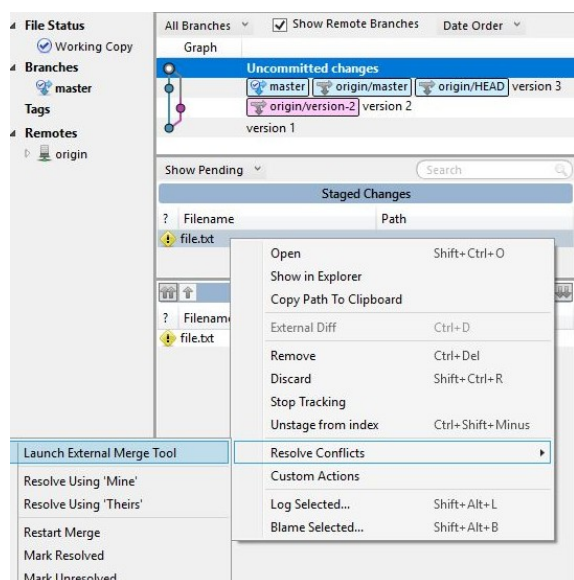


Pokud nemá jedna z větví žádné změny, je situace nejjednodušší. Prostě se „hlava“ (HEAD) větve beze změn posune k hlavě větve se změnami. Tím obě skončí ve stejném stavu. Toto se označuje jako „fast-forward merge“.

Pokud mají obě větve změny, ale žádná ze změn není konfliktní, což znamená, že všechny změny se odehrávaly na rozdílných místech (klidně stejné soubory, ale v obou větvích se měnily jiné řádky), situace odpovídá obrázku výše.

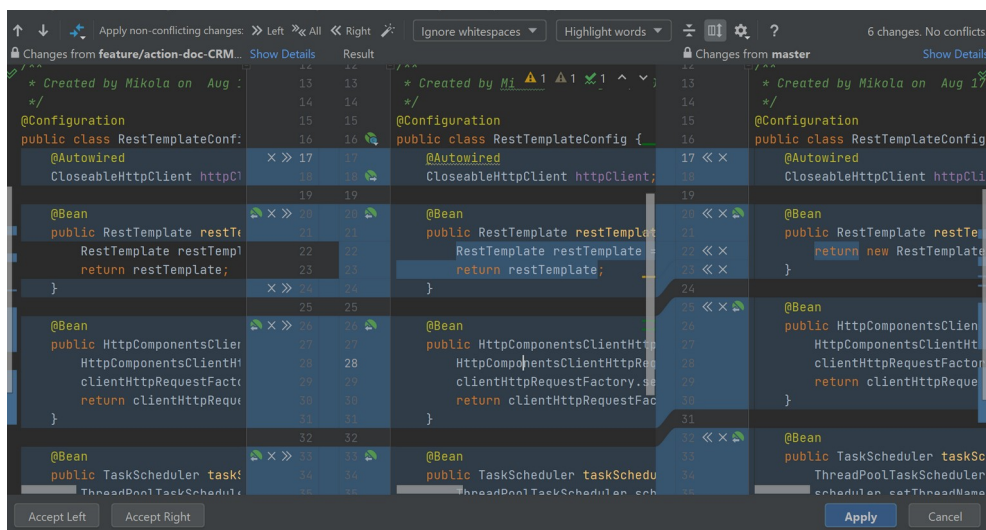
Změny z jedné větve se rozkopírují do druhé větve automatickým commitem a následně obě větve posunou svoji HEAD na nově vytvořený commit. Tím jsou změny obou větví v obou větvích a jsou nyní ve stejném stavu (ukazují na stejný commit).

Pokud mají větve konfliktní změny, je nutné tyto konflikty vyřešit. V této situaci je praktičtější využít program s grafickým rozhraním (Git UI, Sourcetree apod.):



U jednotlivých souborů obsahující konfliktní změny mohu vybrat jednu z verzí souboru a změny z druhé větve zahodit (v obrázku „Resolve using Mine/Theirs“ – použít moji větev nebo druhou).

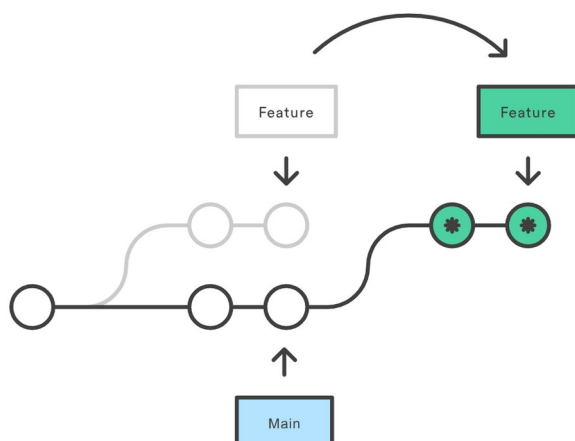
Alternativně mám možnost projít změny a vybrat, které změny z obou větví aplikovat a které zahodit (zde ukázka z git integrace v IntelliJ) – uprostřed výsledný soubor, vlevo jedna verze souboru, vpravo verze z druhé větve:



Tlačítka se šipkami (>>, <<) říkám, že daný změněný blok chci přidat do výsledné verze souboru. Tlačítkem křížku (X) říkám, že změnu v daném bloku chci zahodit.

Zpravidla bývají bloky barevně označené podle situace – zelené jsou nově přidané bloky textu, červeně smazané bloky a modře upravené (částečně přepsané).

Větve se nemusí spojovat pouze příkazem merge. Další možností je posunutí větve na konec jiné větve příkazem **rebase**. Tudíž všechny commity jedné větve se přesunou před nejnovější commit druhé větve, jako by se vytvořily až poté – viz obrázek níže. Rebase se většinou doporučuje jen pro malé množství změn, pro velké změny preferujte příkaz merge.



Nyní už jen zbývá Git vyzkoušet :)

Sestavte skupiny, vyberte si projekt, nastavte si prostředí a začněte paralelní vývoj. Postupně commitujte verze, pushujte je na remote a fetchujte a pullujte verze ostatních z týmu, dokud nebude projekt hotový. Snažte se vyhnout merge konfliktům, a pokud i přeci jen nastanou, vyřešte je, aby někdo nepřišel o svoji práci a projekt po spojení fungoval správně.

Pro vzdálený repositář použijte školní stránky <https://gitlab.spseplzen.cz>

Jeden ze skupiny vytvoří projekt s názvem „VAP2“, viditelnost nastavte na Private a vypněte automatické README:



Create blank project

Create a blank project to store your files, plan your work, and collaborate on code, among other things.

Následně do týmu pozvěte ostatní členy týmu jako roli „Maintainer“ – nejvyšší role pro vývojáře. Stejným způsobem pozvěte i mě (uživatelské jméno „hladilova“):

Project information

Invite your team

Add members to this project and start collaborating with your team.

[Invite members](#)

Vlastník repositáře musí následně repositář nastavit výchozím commitem – vytvoření nového lokálního repositáře (git init), nastavení remote (git remote), commit s výchozím stavem projektu (git commit) a nahrání verze na remote (git push).

Ostatní členové týmu si repositář následně naklonují (git clone).

Všichni si následně vytvoří větev dle svého jména (git branch) a začnou vývoj.